IS2205 - Mobile Application Design & Development

Kotlin Programming

Prerequisites

- Knowing Java OR Willing to learn the main concepts of OOP
- Basic knowledge about IDEs
- Access to a computer and internet connection
- Android device and the USB cable
- Or the emulator

What is Kotlin?

- Kotlin is a modern programming language
- Developed by JetBrains, the creators of IntelliJ IDEA (2011-2012)
- Statically-typed
- Runs on the JVM
- Kotlin is also fully interoperable with Java

Kotlin

- A modern programming language intended for productivity
- Expressive and concise : less boilerplate code
- Safer code
- Interoperable : Java ⇔ Kotlin , Allows gradual transformation

Safe Code

```
fun main() {
   var name: String? = "John"
   // The variable 'name' is nullable since it has a '?' after the type.

   val length = name?.length
   // The safe call operator '?' checks if 'name' is null.
   // If null, the expression evaluates to null without a NPE
   // otherwise returns the length of ‘name’
   println("Length of the name: $length")
}
```

Simple Kotlin program

What ***code editor*** we can use?

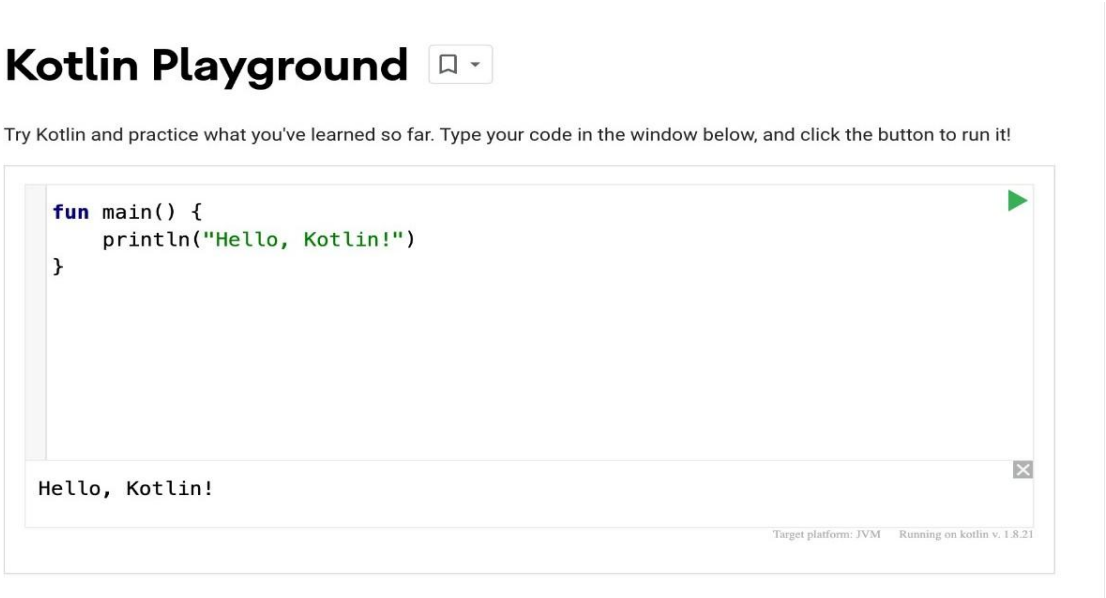
Interactive editors : Kotlin Playground

- Can practice the basics of the language
- Web browser based and no need to install
- Running code is supported

Installable IDEs : Android Studio,VSCode What are the features of these?

Simple Kotlin program…

Kotlin Playground : Link



Kotlin Compiler

Create a simple application in Kotlin that displays `"Hello, World!"`. In your favorite editor, create a new file called `hello.kt` with the following lines:

```
fun main() {
    println("Hello, World!")
}
```

Compile the application using the Kotlin compiler:

```
kotlinc hello.kt -include-runtime -d hello.jar
```

The `-d` option indicates the output path for generated class files, which may be either a directory or a **.jar** file. The `-include-runtime` option makes the resulting **.jar** file self-contained and runnable by including the Kotlin runtime library in it.

To see all available options, run

```
kotlinc -help
```

Run the application.

```
java -jar hello.jar
```

Demo

Functions

Question: What is a function?

Functions in C++

Declaration, definition and call?

```
// Function declaration  (Prototype)
int addNumbers(int a, int b);
```

```
// Function definition
int addNumbers(int a, int b) {
    return a + b;
}
```

```
// Main function  int
main() {
    int num1 = 5;  int
    num2 = 10;

//  Function call
    int sum =
addNumbers(num1, num2);
    cout << "Sum: " << sum
<< endl;  return 0;
}
```

Functions in Kotlin

Declaration and definition are same in Kotlin

```
fun main(){
    println("Hello, world!")

    kotlinHello()
}

fun kotlinHello(){
    println("Hello, Kotlin!")

}
```

```
Hello, world!
Hello, Kotlin!
```

Functions…

Revisit later

Styling tips

Code style is usually part of the development culture of the language
Here are some of the relevant styling recommendations for Kotlin:

- ❏ Function names should be in camel case and should be verbs or verb phrases.
- ❏ Each statement should be on its own line **Note**: This is a language restriction
- ❏ The opening curly brace should appear at the end of the line where the function begins.
- ❏ There should be a space before the opening curly brace.

Styling tips…

space

```
fun main() {
    println("Hello, world!")
}
```

aligned vertically

```
fun main() {
    println("Hello, world!")
}
```

indent by 4 spaces

```
fun main() {
    println("Hello, world!")
}
```

Questions

Assume you do not want to follow the recommended style in Kotlin :

Where/How can this be acceptable practice?

Where/How can this be unacceptable practice?

Main method with arguments…

```
fun main(args: Array<String>) {
    println("Hello, World!")
}

fun main (args : Array<String>){

    for (arg in args)
        println("Arg: $arg")

}
```

Exercises

Rewrite the following 2 Kotlin programs by correcting the errors:

```
//Program 1
fun main() {
      println("Hello, world!')

}
//Program 2
fun main() {

   print("Hello, world!") print(" from Kotlin!")

}
```

Question

Name other languages that are statically-typed

Name languages that is dynamically typed

Data Types

Kotlin data type	What kind of data it can contain	Example values
String	Text	"Add contact","Search","Sign in"
Int	Integer number	32,1293490,-59281
Double	Decimal number	2.0,501.0292,-31723.9999
Float	Decimal number (that is less precise than a Double).	5.0f,-1630.209f,1.294078F
Boolean	true or false. Use this data type when there are only two possible values. true and false are keywords in Kotlin.	true , false

Data Types - Primitive ? Wrapper?

“Kotlin does not differentiate Wrapper class data types and primitive data types as Java does”

Type difference Java/Kotlin

1. In Java, you have both the primitive type int and the corresponding wrapper class Integer.

2. In Kotlin, there is no distinction between primitive types and wrapper classes.

3. This design choice in Kotlin eliminates the need for explicit boxing and unboxing operations that are required in Java

Identifying Data Type

```
var name = 'H'

println(name)  //Output : value stored in the variable

println(name::class.simpleName)

// Output: class name = Char

println(name::class.qualifiedName)

// Output: qualified class name kotlin.Char
```

Variables and Values

Powerful type inference - Let the compiler infer the type

You can explicitly declare the type if needed

Kotlin is a statically-typed language.

The type is resolved at compile time and never changes.

Variables and Values in Kotlin

Variable/Value definition(declaration is no difference in Kotlin) has 3 options

1. Explicit type definition and initialization

2. Explicit type definition but no initialization

3. Implicit type definition and initialization

Variables and Values in Kotlin…(1)

1. Explicit type definition and initialization

```
val count: Int = 2
```

Variables and Values in Kotlin...(2)

2. Explicit type definition but no initialization

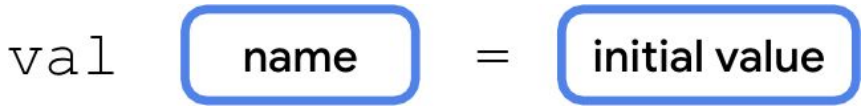
```
val count: Int
```

Important: Once a type has been assigned by you or the compiler, you can't change the type or you get an error.

Variables and Values in Kotlin…(3)

3. Implicit type definition and initialization (A.K.A. Type inference)

```
val count = 3
```



Example

```
fun main() {
    val spamCount: Int = 4
    val newMessages: Int
    newMessages = 2 //initilization
    val unreadCount = 15
    val readCount = 100
    println("You have ")
    print("$newMessages new messages ")
    println("and $unreadCount unread messages")
    println("There are ${unreadCount + readCount} total messages in your inbox
    println("Delete $spamCount Spam messages?[Y/n]")
}
```

```
You have
2 new messages and 15 unread messages
There are 115 total messages in your inbox
Delete 4 Spam messages?[Y/n]
```

Exercise

Execute the following programs in Kotlin playground and write the error messages thrown out. Identify 2 different methods to fix the error.

```
//program 1

fun main() {

    val count

    count = 9


    println("You have $count unread messages.")

}
```

Variables and Values in Kotlin…

Note that values (val) cannot be reassigned

However if we define it as a variable (var) then we can re-assign the value

```
fun mail1() {
    val count = 0
    println("You have $count unread messages.")
    count = 9
    println("You have $count unread messages.")
}


fun mail2() {
    var count = 0
    println("You have $count unread messages.")
    count = 9
    println("You have $count unread messages.")
}
```

```
Val cannot be reassigned
```

Variables: updating the value

```
var count = 1

Increment of a variable is possible from C++/Java convention
count = count + 1 // count is 2 now

count++ // count is 3 now
```

Null safety

- In Kotlin, variables cannot be null by default

Declare an Int and assign null to it.

```
var numberOfBooks: Int = null
```

⇒ error: null can not be a value of a non-null type Int

Safe call operator

The safe call operator (?), after the type indicates that a variable can be null.

```
//Declare an Int? as nullable

var numberOfBooks: Int? = null
```

In general, do not set a variable to null as it may have unwanted consequences.

Testing for null...default

- Check if the numberOfBooks variable is not null. Then decrement that variable.

```
fun main(args : Array<String>) {
    var numberOfBooks: Int = 6 //var is not nullable
    if (numberOfBooks != null) { //old method
        println("A")
         numberOfBooks = numberOfBooks.dec()
    }
```

New Kotlin way of writing it, using the safe call operator.

```
    numberOfBooks = numberOfBooks.dec() //if var is not nullable use direct call
    println("B")
    print("numberOfBooks=$numberOfBooks")
}
```

Testing for null...with nullable

- Check if the numberOfBooks variable is not null. Then decrement that variable.

```
fun main(args : Array<String>) {
    var numberOfBooks: Int? = 6 //var is nullable
    if (numberOfBooks != null) { //old method
        println("A")
         numberOfBooks = numberOfBooks.dec() //this is allowed
    }
```

New Kotlin way of writing it, using the safe call operator.

```
    numberOfBooks = numberOfBooks?.dec() //if var is nullable use safe call
    println("B")
    print("numberOfBooks=$numberOfBooks")
}
```

Operators : Logical

Operator	Operation	Sample
&&	Logical AND	x < 5 && x > 0
||	Logical OR	x < 5 || x > 10
!	Logical NOT	x = !x

Operators : Comparison

Operator	Operation
==	equal to
!=	not equal to
>	greater than
<	less than
>=	gt or equal
<=	lt or equal

Operators : Assignment…Fill the remaining

Operator	Operation	Expansion
=	assign a copy	
x+=5		x=x+4
-=	subtract and assign	
*=		
/=		
%=	assign the remainder	x=x%5

Strings

Strings are any sequence of characters enclosed by double quotes.

```
val s1 = "Hello world!"
```

String literals can contain escape characters

```
val s2 = "Hello world!\n"
```

Or any arbitrary text delimited by a triple quote (""")

```
val text = """  var bikes = 50  """
```

Strings…

Kotlin **Strings** can be considered as character arrays.

```
var txt = "Hello World"

println(txt[0]) // first element (H)

println(txt[2]) // third element (l)
```

String indexes start with 0.

Length of a String can be found with the length property.

```
println(txt.length) // 11
```

Strings...search

Kotlin **Strings** have a search function to the start index.

```
var txt = "For trump safety concerns trumped financial concerns"

println(txt.indexOf("trump"))  // Outputs 4
```

String templates

```
val numberOfDogs = 3

val numberOfCats = 2
//concatenation
val msg = "I have $numberOfDogs dogs" + " and $numberOfCats cats"

println(msg)
//expression inside the template
print("I have ${numberOfDogs+numberOfCats} cats and dogs")
```

String methods …

String manipulation methods are in-built to the String class as in Java

String.toUpperCase() - Convert String to uppercase

String.toLowerCase() - Convert String to lowercase

String.compareTo(String) - returns 0 if two Strings are equal

+ operator OR String.plus() - String concatenation

Conditional Statement

```
if (condition1) {
  // if condition1 is true
} else if (condition2) {
  // if the condition1 is false and condition2 is true
} else {
  // if both condition1 and condition2 are false
}
```

Conditional Statement - Returns a Value

```
val age = 17

val decision = if (age < 18) {

  "No"

} else {

  "Yes"

}

println("Can I watch any movie? "+decision )
```

When Expression

```
val number = 3

val result = when (number) {
  1 -> "Foo"
  2 -> "Bar"
  3 -> "Whiz"
  else -> "Invalid name." //Mandatory
}

println(result)
```

When Expression … Some examples

```
val text = "mycat"

val result = when (text) {
  "mycat" -> "Rasputin"
  "hiscat" -> "Blacknight"
  else -> "not a cat"
}

println("My cat is "+result)
```

When Expression … Some examples

```
val text = "mycat"
val result = when (text) {
  "mycat" -> {
        println("Selecting Rasputin")
        "Rasputin"
}
  "hiscat" -> "Blacknight"
  else -> "not a cat"
}
println("Selected $result")
```

Loops : While,Do While

Kotlin While, and Do while Loops are comparable to those of Java

Exercise:

Implement above loop constructs in Kotlin Playground

Question

If I have a val list of cars, can I change the first car in the array?

More Kotlin Concepts

For loop...

```
val carMakers = arrayOf("Toyota", "BYD", "Nissan", "Honda")

for (x in carMakers ) {

  println(x)

}

for (i in carMakers.indices ) {

  println(carMakers[i])

}
```

Note : C style - index based - for loop is not supported by Kotlin

Arrays

```
val cars = arrayOf("Toyota", "BYD", "Nissan", "Honda")

cars[0] = "BMW" //replacing an array element

println(cars[0])


val cars = arrayOf("Toyota", "BYD", "Nissan", "Honda")

println(cars.size) // size is a property
```

Exercise

You are given the following array. Write code to search the string "BMW" in the array.

```
val carMakers = arrayOf("Toyota", "BYD", "Nissan","BMW", "Honda")
```

Arrays…with ‘in’ operator

```
val cars = arrayOf("Toyota", "BYD", "Nissan", "Honda")

if ("BYD" in cars) {
  println("It’s here!")
} else {
  println("No it is not!")
}
```

Arrays…

```
val cars = arrayOf("Toyota", "BYD", "Nissan", "Honda")

cars.set(0, "Geely")  // set the first element equal to "Geely"

cars.set(3, "Tesla")   // set the second element…

println(cars.get(0)) // retrieve the first element

println(cars[1]) // print the second element using []
```

Ranges

```
for (chars in 'a'..'x') { //or 'A'..'Z' or 0..9

  println(chars)

}
```

These simple ranges work only in the ascending order.

Ranges...

```
for (num in 0..10) { //ascending order

  println(num)

}

for (num in 10 downTo -10) { //descending order

  println(num)

}
```

Ranges...

```
//Stepping

for (num in 10 downTo 2 step 2) { //descending order

  println(num) //step : reduce by two

}
```

Note: Both limits are included unless we step over.

Exercise

```
/What happens if the step is 3?

for (num in 10 downTo 2 step 3) {

  println(num) // output ?

}
```

Arrays and ranges

```
val carMakers = arrayOf("Toyota", "BYD", "Nissan", "Honda")

//combining with range specification

for (index in 0..carMakers.size-1) {

  println(index)

}
```

Ranges...

```
//continue and break

for (nums in 5..15) {
  if (nums == 10) {
    continue //continue try the next item; break ends the loop
  }
  println(nums)
}
```

Functions revisited…common syntax

```
fun functionName(para1: d_type1, para2: d_type2,….): return_type{
    //code of the function
    return value_of_return_type
}
```

Note : Compare with the variable definition

Functions...

```
fun myFunction(x: Int, y: Int): Int {
  return (x + y)
}
fun main() {
  var result = myFunction(3, 5)
  println(result)
  }
```

Functions...

```
fun myFriend(){
  println("Am I doing fine?!") // no explicit return
}
fun main() {
  var result = myFriend()
  println("Yay! This function just executed!")
  println(result) // there is a default return type
}

//kotlin.Unit
```

Functions - Default Arguments

```
fun main() {
  // Call with argument
  greet("Guten tag!") // Output: user supplied
  // Call without argument
  greet() // Output: default
}

fun greet(greeting: String = "Good day!") {
    greet("$greeting!")
}
```

Functions - Named Arguments

```
fun main() {
  // Call with argument
  greet(greeting="Guten tag", name="Wijey!") // Output: user supplied
  // Call without argument
  greet() // Output: default
}

fun greet(greeting: String = "Good day",name: String = "to you!") {
    println("$greeting $name")
}
```

Ternary Operator

(condition) ? <expression_true> : <expression_false> ;

What is the replacement for the ternary operator in Kotlin dialect?

Conditional Block that Returns...

Java	(condition)	?	<expression_true>	:	<expression_false>;
Kotlin	(condition)	if	<expression_true>	else	<expression_false>
"	(condition)	if	{<expression_true>}	else	{<expression_false>}

```
val a = 7
val b = 8
val min:Int //Following is an 'if' condition as an expression
val max = if (a > b) {min=b;a} else {min=a;b} // code block or an expression
print("Max:$max,Min:$min") //can be included
```

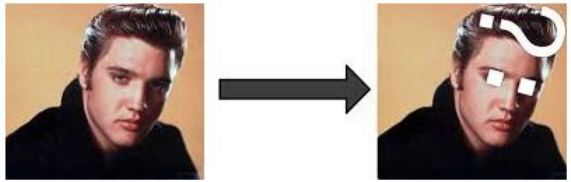
?: Elvis Operator (!= Ternary Operator)

```
var name: String? = "Seth"
var address: String? = null

var len1: Int = name?.length ?: -1
var len2: Int = address?.length ?: -1

println("Length of name is $len1") // 4
println("Length of address is $len2") // -1
// Useful for replacing a default value on null return
```

The name Elvis Operator comes from the famous American singer **Elvis Presley**. His hairstyle resembles a Question Mark



Source: Wojda, I. Moskala, M. Android Development with Kotlin. 2017. Packt Publishing

Break

OOP Concepts

Classes

```
class Car {
  var brand = "Toyota"
  var model = "aqua"
  var year = 2014
 }
```

Objects

```
val c1 = Car()

c1.brand = "Honda"

c1.model = "Civic"

c1.year = 2011

val c2 = Car()

c2.brand = "Nissan"

c2.model = "Bluebird"

c2.year = 2007

println(c1.brand+" "+ c2.brand)  // Honda , Nissan
```

Constructor Creates Objects…

```
class Car(var brand: String,
var model: String, var year:
Int)


fun main() {

  val c1 = Car("Honda",
"Civic", 2011)

 }
```

Constructors...

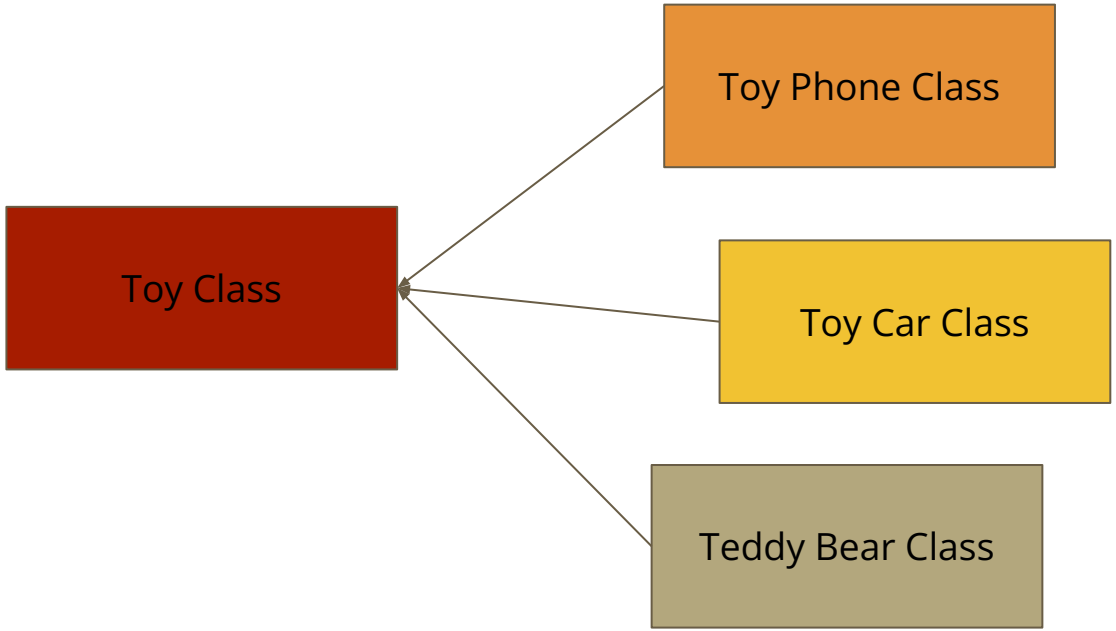
```
class Car(var brand: String, var model: String, var year: Int) {
  init {//initializing block : multiples can appear and execute in that order
    println("model is $model")
  }
} //Note : constructor variables must have explicit type definitions


fun main() {
  val c1 = Car("Honda", "Civic", 2011)
}
```

Member Functions/Instance Methods/Class Functions

```
class Car(var brand: String, var model: String, var year: Int) {
  fun start() {//class function
    println("Brrrr…m!!")
  }
}
val c1 = Car("Honda", "Civic", 2011)
c1.start() // Call the function
}
```

Inheritance



Is a relationship

Inheritance...

```
open class MyParentClass {// Superclass : closed by default
  val parentsAsset = 5
}
class MyChildClass: MyParentClass() {// Subclass : Compare with var and fun definition
  fun myFunction() {
    println(parentsAsset) // parentsAsset is now inherited from the superclass
  }
}
val myObj = MyChildClass() // Create an object of MyChildClass and call myFunction
myObj.myFunction()
}
```

Exercise

Develop a class with Java/C++ programming Language, with at least two class functions/member functions.

Convert the above Class to Kotlin syntax.

Exception Handling

Why Exception Handling?

Error Handling vs Exception Handling

Checking for : Mixed with program flow vs Separated into try-catch blocks

Propagation : Manual via return codes vs Automatic propagates up the stack

Generalization : Possible vs Difficult

Exception Handling...catching

```
fun main() {
    try {
        val num = 10 / 0
        println(num)
    } catch (e: ArithmeticException) {
        println("Divide by zero is not allowed")
    }
}
```

Exception Handling...finally(ing)

```
fun main() {
    try {
        val num = 10 / 0
        println(num)
    }finally{ //Note : either catch or finally must accompany a try
        println("Division end")
    }
}
```

Exception Handling...throw(ing)

```
fun main(args: Array<String>) {
    try {
        val num = args[0] ?: throw IllegalStateException("Parameter required")
        println(num)
    }finally{
        println("Parsing complete")
    }
}
```

Thank You!

Reference

https://kotlinlang.org/docs/getting-started.html

https://play.kotlinlang.org/byExample/overview